MotoTrack — Rencana Akhir & Laporan Proyek

Pertanyaan yang dijawab: dari video motovlog ego-POV, apakah motor bisa nyalip ke depan atau tidak?

Status: selesai dan ter-deploy. 118 tes lulus di `github.com/rynnfrds/MotoTrack`.

Tanggal: 12 November 2026

Klip default: footage ego-POV portrait dari YouTube Short publik (`youtube.com/shorts/5zzcCtHc3xg`). Bukan lisensi terbuka, tidak ikut ke repo.

---

1. Ringkasan eksekutif

MotoTrack menerima klip motovlog, mendeteksi dan melacak pengguna jalan, memproyeksikan koridor ego, mengukur ruang lateral bebas di dalamnya, lalu memberi vonis per momen: `SAFE` / `TIGHT` / `BLOCKED` / `UNKNOWN`.

Sistem berjalan penuh di CPU tanpa GPU. Waktu proses ~65 ms per frame yang dianalisis, sekitar 5,9 fps. Klip 37 detik selesai dalam ~63 detik.

Yang penting dipahami: **deteksi objek bukan kontribusi proyek ini** — itu sudah diselesaikan orang lain dan kita pakai bobot siap pakai. Kontribusi aslinya ada di lapisan keputusan: geometri koridor, normalisasi gap yang bebas kalibrasi, dan ambang yang diturunkan dari data berlabel, bukan ditebak.

---

2. Keputusan model (hasil riset)

Sebelum membangun, saya riset dulu apakah sudah ada model siap pakai. Hasilnya:

Kebutuhan	Yang tersedia	Keputusan
Deteksi kendaraan	YOLO26n (COCO), rilis v8.4.0	Dipakai, tanpa training
Deteksi lalu lintas campuran	fine-tune `visaitech/vehicle-mixed-traffic-yolo26n`	Dipakai sebagai alternatif
Estimasi kedalaman	`yolo26n-depth.pt`	Diunduh, tidak dipakai di jalur utama
Model "bisa nyalip?"	**tidak ada**	**dibuat sendiri: logika aturan + geometri**
Dataset POV Indonesia	**tidak ada**	pakai footage motovlog publik (YouTube Short)

Catatan penting: **YOLO27 belum rilis.** Dokumentasi resmi menyatakan model belum tersedia dan belum ada tanggal peluncuran. Jadi YOLO26n adalah pilihan terbaru yang nyata, dan di CPU ia ~43% lebih cepat dari YOLO11n.

Konsekuensi yang jujur: tidak ada satu pun bobot yang dilatih di proyek ini. Semua checkpoint pihak ketiga, berlisensi AGPL-3.0.

---

3. Arsitektur

```
video  ->  deteksi + tracking (YOLO26n)
       ->  estimasi jarak (pinhole, asumsi tinggi objek + HFOV 78 derajat)
       ->  koridor ego (trapesium, koordinat relatif)
       ->  uji penghalang: tumpang tindih horizontal kotak dengan koridor
       ->  gap lateral bebas / lebar median kendaraan SEBANDING
       ->  vonis per frame (aturan + override)
       ->  stabilisasi temporal (median 2 s + histeresis 0,5 s)
       ->  annotated.mp4 + events.json + summary.json + timeline.png
```

Tiga keputusan desain yang menentukan

a. Koridor dalam koordinat relatif. Trapesium didefinisikan sebagai fraksi lebar dan tinggi frame, jadi video 16:9 dan reels 9:16 memakai satu jalur kode yang sama. Ini yang membuat klip default portrait 720x1280 jalan tanpa cabang kode terpisah.

b. Gap dinormalisasi lebar kendaraan, bukan piksel.

```
gap_ratio = gap_bebas_px / lebar_median_kendaraan_px   (pada band kedalaman yang sama)
```

Karena pembagi dan pembilang diskalakan oleh perspektif yang sama, rasionya **invarian skala**. Artinya: tidak perlu kalibrasi kamera sama sekali. Ini yang membuat proyek bisa jalan di video apa pun tanpa setup.

c. Penghalang diuji lewat tumpang tindih, bukan titik tengah. Versi pertama menguji apakah titik tengah bawah kotak masuk koridor. Itu cukup untuk dashcam menyusuri lajur, tapi salah untuk ego-POV motor yang menyelip di antara dua truk: titik tengah tiap truk jatuh di luar koridor sempit, keduanya dibuang, koridor disebut kosong, dan vonisnya SAFE padahal celahnya sekitar 1,5 m. Sekarang yang diuji adalah apakah **rentang x kotak beririsan** dengan koridor pada baris kontak tanah objek, ditambah cek horizon supaya gantry dan dek jembatan di atas jalan tidak ikut dihitung.

---

4. Ambang: diturunkan, bukan dikarang

Ambang awal (1,6 / 1,2) adalah tebakan saya. Itu tidak layak dikirim, jadi saya buat `scripts/calibrate.py` yang menyapu 780 pasangan ambang terhadap gold set berlabel manual.

Aturan pemilihannya sengaja asimetris: pilih pasangan dengan kesepakatan tertinggi **dengan syarat tidak pernah menyebut SAFE pada frame yang manusia sebut BLOCKED**.

Pada gold set 30 frame yang sekarang, sapuan itu tidak menghasilkan satu pemenang tunggal. Ia menghasilkan **dataran (plateau)**: semua `safe_ratio` dari 1,0 sampai 1,8 dengan `tight_ratio` 0,4 memberi skor identik. Ambang yang dikirim tetap 1,8, ujung paling konservatif dari dataran itu, karena data tidak bisa membedakan kandidat di dalamnya dan ambang lebih rendah berarti menyebut koridor lapang dengan bukti lebih sedikit.

Ambang	Akurasi 3 kelas	Akurasi biner	SAFE padahal BLOCKED
Tebakan (1,6 / 1,2)	56,7%	100,0%	0
Dikirim (1,8 / 0,4)	**53,3%**	**100,0%**	**0**

Angka di tabel ini dihitung atas 30 baris dengan koridor kosong dihitung SAFE, jadi denominatornya beda dari tabel detektor di bawah (24 baris dinilai). Keduanya disebutkan, bukan dipilih yang paling bagus.

Alasannya: SAFE palsu mengundang pengendara mengambil manuver yang tidak bisa dibatalkan. BLOCKED palsu hanya membuang waktu. Semua keputusan smoothing condong ke arah aman.

---

5. Hasil pengukuran

Kondisi: 1 klip (`clip_001.mp4`), 720x1280 portrait, 29,92 fps, 37,22 detik, 1113 frame. Gold set: 30 frame berlabel manual oleh satu orang. CPU saja.

Perbandingan detektor

Detektor	Akurasi biner	Akurasi 3 kelas	Dinilai	UNKNOWN
`coco`	**100,0%**	**41,7%**	24	6
`mixed`	74,1%	37,0%	27	3

Peringkatnya **berbalik** dibanding klip sebelumnya: dulu `mixed` menang, sekarang `coco`. Itu justru intinya — pemenang ditentukan footage, bukan diasumsikan.

Yang TIDAK berhasil

Akurasi 3 kelas 41,7% **belum layak pakai.** Recall TIGHT hanya 6,7%: 14 dari 15 frame sempit disebut BLOCKED. Sistem condong bilang BLOCKED — aman, tapi tidak berguna.

Angka 100% biner itu harus dibaca bersama konteksnya: ia berarti aturan tidak pernah berbeda pendapat dengan pelabel pada pertanyaan SAFE vs bukan-SAFE di 24 frame, dan sebelum bug

koridor diperbaiki angkanya 45,8%. Lompatannya bukti perbaikan itu nyata, sekaligus bukti angka ini rapuh dengan 30 frame.

Angka ini ditulis apa adanya di `docs/METRICS.md`. Ambang di-fit **pada 30 frame yang sama** yang dipakai menilai, jadi hasilnya **optimistis**, bukan validasi.

---

6. Bug yang ditemukan dan diperbaiki

Ini bagian yang paling menunjukkan cara kerja sebenarnya.

#	Bug	Dampak terukur	Perbaikan
1	Masker bodywork motor sendiri pakai ambang bawah frame saja	Menelan mobil 1,5 m di depan — objek paling penting	Butuh dua syarat: menempel bawah **dan** >=75% lebar frame
2	`gap_ratio` dinormalisasi lebar median **semua** deteksi	Satu box jauh 30 px memperkecil pembagi, rasio menggelembung	Pembagi diambil dari band kedalaman yang sama
3	`UNKNOWN` diikutkan dalam median	76,9% frame jadi UNKNOWN	UNKNOWN dikecualikan (artinya "tidak berpendapat")
4	Frame yang dilewati stride ditulis mentah	5 dari 6 frame output tanpa overlay di 60 fps → kedip	Digambar dengan state terakhir
5	Menahan state terakhir membekukan overlay	67% pasangan frame identik → box melompat	Buffer + interpolasi antar dua state → 0,2%
6	Track hilang-timbul 1-2 sampel	56 event hilang-muncul	EMA sudut box + coasting 0,4 s
7	`/api/health` melaporkan port default	Info palsu saat desktop pindah port	Melaporkan port yang benar-benar terikat
8	Uji penghalang pakai **titik tengah bawah** kotak	Di klip portrait, truk di kiri-kanan dibuang, koridor disebut kosong, vonis SAFE padahal pengendara menyelip di celah	Uji **tumpang tindih horizontal** kotak dengan koridor + cek horizon

#	Bug	Dampak terukur	Perbaikan
		1,5 m	

Bug 8 adalah yang paling penting: itu satu-satunya di daftar ini yang mengubah akurasi, dan ketahuannya justru karena klip default diganti. Akurasi biner `coco` naik dari 45,8% ke 100,0%, `mixed` dari 37,0% ke 74,1%.

Bug 4–6 semuanya **soal tampilan, bukan akurasi** — dan itu dinyatakan eksplisit di dokumentasi. Mudah sekali membuat demo terlihat lebih tenang lalu menyimpulkan modelnya jadi lebih pintar. Bukan.

Setiap bug punya tes regresi. Saya buktikan tes itu benar-benar menangkap: dengan bug dikembalikan, tes gagal (mis. `19.4 > 10.0`, rasio frame bergerak jatuh ke 15%).

---

7. Yang dibangun

Komponen	Isi
Pipeline	deteksi, tracking, geometri, vonis, stabilisasi, output
CLI	`analyze`, `preview-corridor`, `eval`, `compare-detectors`, `web`, `desktop`
Web app	upload, progress nyata, halaman hasil, timeline, export JSON
Aplikasi Windows	`Start MotoTrack.bat` → jendela native, tanpa browser
Evaluasi	gold set, matriks konfusi, kalibrasi ambang, `measure_display`
Notebook Colab	fine-tune opsional, data-prep terverifikasi
Dokumentasi	README, model card, metrics, demo runbook
Tes	118 tes

Port

Default **8770**. Port 8000 sengaja dihindari karena dipakai aplikasi lain di mesin ini. Bisa dioverride via `--port` atau `MOTOTRACK_PORT`. CLI memeriksa port dulu dan menyarankan alternatif alih-alih gagal separuh jalan. Aplikasi desktop pindah port otomatis. Tidak ada proses lain yang pernah dimatikan.

---

8. Batasan yang harus dibaca sebelum dipakai

Ini bukan sistem keselamatan. Jangan dipakai untuk berkendara sungguhan.

Situasi	Perilaku
Malam, hujan, terowongan, silau	Deteksi kolaps → `UNKNOWN`, bukan menebak
Motor miring saat menikung	Koridor tetap di frame, jalan keluar dari koridor
Lalu lintas lawan arah	Tidak dimodelkan; kendaraan lawan yang masuk koridor dihitung penghalang, jadi condong aman
Kendaraan mau pindah lajur	Tidak dimodelkan, tidak ada prediksi gerak
Dua truk mengapit di kiri dan kanan	Terdeteksi sebagai dua interval terblokir dengan satu celah sempit; inilah kasus yang membongkar bug titik-tengah-bawah
Bus dianggap mobil	Kelas `4-wheeler` menyatukan sedan–bus, tinggi asumsi salah
Pejalan kaki	**Data latih hanya 30 instance vs 731 motor** — paling lemah di objek paling berbahaya

Jarak bersifat estimasi monokular dari asumsi tinggi objek dan HFOV 78 derajat — bukan pengukuran sensor. Dilaporkan sebagai band (dekat <3 m, sedang <8 m, jauh), dengan peringatan di HUD.

---

9. Kalau dilanjutkan, urutan prioritasnya

Prioritas	Langkah	Alasan
1	Tambah klip sampai gold set ~150 frame	30 frame terlalu sedikit; 41,7% itu optimistis
2	Pisahkan data fit dan data uji	Sekarang ambang di-fit pada set yang sama yang dinilai
3	Tangani lean/miring	Penyebab gagal paling nyata di footage nyata
4	Fine-tune untuk pejalan kaki	30 instance tidak cukup untuk objek paling berbahaya
5	GPU atau `imgsz` lebih kecil	Deteksi 10 fps, bukan realtime

Langkah 1 dan 2 paling berdampak. Sisanya menambah kompleksitas sebelum dasar ukurannya cukup kuat.

10. Cara menjalankan

Aplikasi Windows (tanpa browser, tanpa terminal)

```
uv pip install --python .venv\Scripts\python.exe pywebview   (sekali saja)
double-click Start MotoTrack.bat
```

Browser

```
.venv\Scripts\python.exe -m mototrack.cli web
-> http://127.0.0.1:8770
```

CLI

```
.venv\Scripts\python.exe -m mototrack.cli analyze klip.mp4 --detector coco
.venv\Scripts\python.exe -m mototrack.cli compare-detectors
.venv\Scripts\python.exe scripts\calibrate.py --detector coco
.venv\Scripts\python.exe scripts\measure_display.py
.venv\Scripts\python.exe -m pytest tests
```

Kalau perbaikan kode tidak terlihat

Uvicorn tanpa `--reload` menyimpan kode yang ia mulai. Restart server **dan** hard-refresh browser (Ctrl+Shift+R). Keduanya perlu; salah satu saja menyisakan perilaku lama.

---

11. Lisensi & atribusi

Paket Ultralytics dan kedua checkpoint berlisensi **AGPL-3.0**. Proyek ini mewarisi kewajiban itu: kalau suatu saat dijadikan layanan jaringan, source harus tersedia.

Bobot tidak di-vendor ke repo; diunduh `scripts/fetch_models.py` ke `models/` yang di-gitignore. Footage dan gold set juga tidak masuk repo — bukan milik kita untuk diredistribusi.

Klip default adalah footage pihak ketiga dari YouTube Short publik (`youtube.com/shorts/5zzcCtHc3xg`, "macet nyalip motovlog viral"). **Bukan lisensi terbuka.** Karena itu ia tidak ikut ke repo, dan indeks frame di `data/gold/gold.csv` merujuk ke satu berkas persis. Siapa pun yang mau mengulang pengukuran harus menyediakan klipnya sendiri lalu melabeli ulang gold set.

Dataset fine-tune opsional: `visaitech/vehicle-mixed-traffic-detection`, CC-BY-4.0, 239 frame latih / 54 validasi.

---

12. Kesimpulan

Pertanyaan "bisa nyalip atau tidak" **tidak bisa dijawab model siap pakai**, karena model semacam itu tidak ada. Jadi jawabannya dibangun: deteksi dipinjam, keputusan dibuat sendiri, ambangnya diturunkan dari data.

Hasilnya berjalan end-to-end di CPU, di web dan sebagai aplikasi Windows. Angkanya ditulis apa adanya, termasuk yang buruk. Delapan bug ditemukan lewat pengukuran, bukan lewat asumsi — dan tiap perbaikan punya tes yang terbukti menangkap regresi.

Bug kedelapan layak disebut terpisah. Ia tidak ditemukan dengan membaca kode, tapi dengan **mengganti klip default**: begitu footage-nya jadi portrait ego-POV yang sesungguhnya, uji penghalang berbasis titik tengah kotak membuang truk di kiri dan kanan, lalu menyebut koridor kosong padahal pengendara sedang menyelip di celah 1,5 m. Akurasi biner naik dari 45,8% ke 100,0% setelah diperbaiki. Itu sekaligus bukti bahwa klip uji yang lebih dekat ke domain aslinya menemukan cacat yang tidak terlihat pada footage dashcam.

Nilai portofolio proyek ini bukan pada angka akurasinya, tapi pada caranya mengukur: menyatakan asumsi, mem-fit ambang ke data berlabel, melaporkan hasil negatif, dan membedakan "tampilan lebih halus" dari "model lebih pintar".